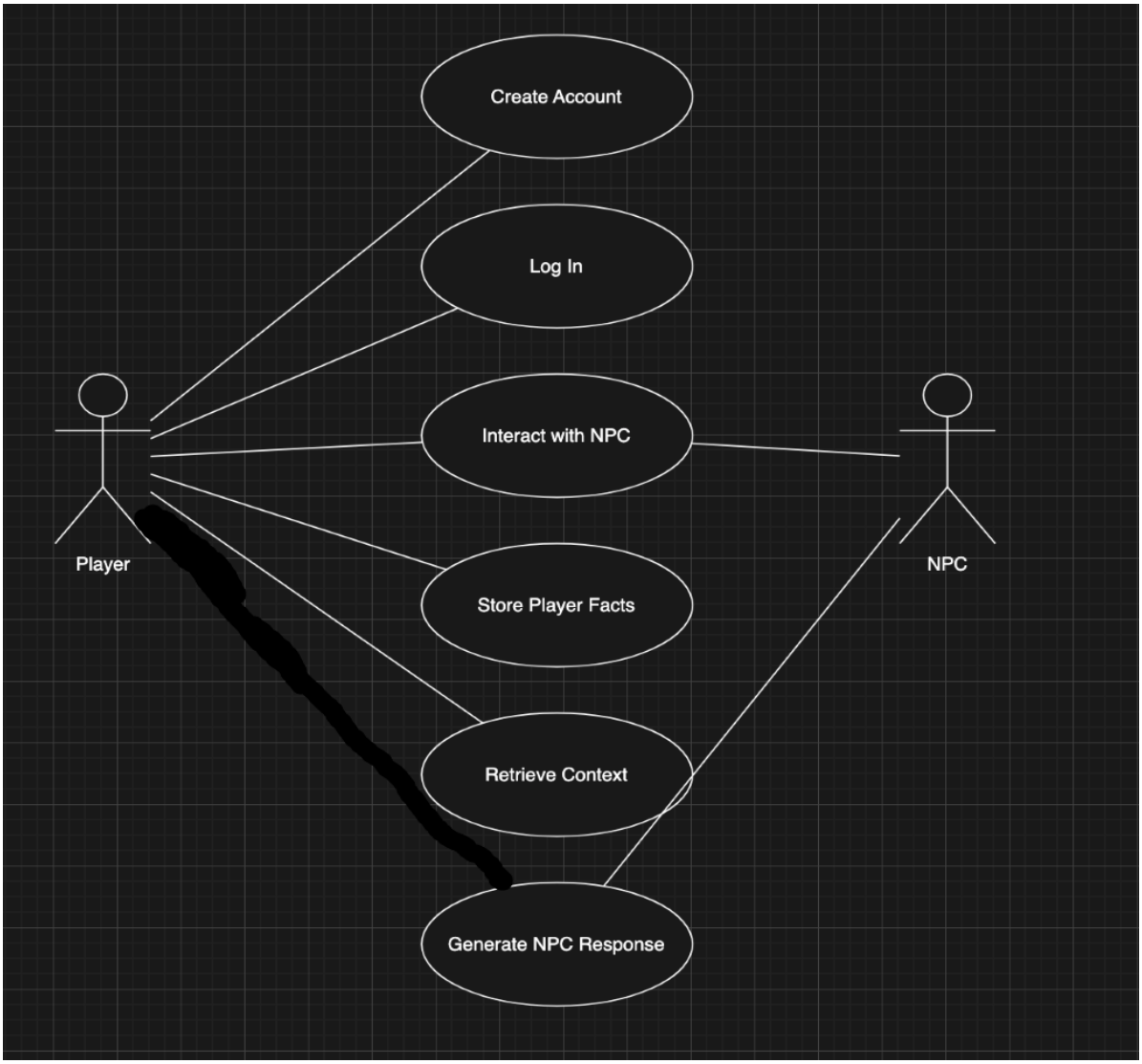
SOFTWARE ENGINEERING
Fall 2026, Georgia State University
Date: 09/24/2026
Team 08: Joshoua Hossain, Sweeya Ghanta, Arturo Hernandez, Jomari Ingol

Introduction: This document specifies the use case, requirements, architecture of the application for "Agent Appetite," a browser-based restaurant simulation game. The core innovation is the integration of Large Language Models (LLMs) to power Non-Player Characters (NPCs) that possess persistent memory and emotional states, reacting to player actions based on past interactions rather than pre-coded scripts.

USE CASE DIAGRAM:

This use case diagram is a visual representation of how external actors (users or systems) interact with a system's core functionalities to achieve specific goals, Player & NPC's roles.



SOFTWARE ENGINEERING
Fall 2026, Georgia State University
Date: 09/24/2026
Team 08: Joshoua Hossain, Sweeya Ghanta, Arturo Hernandez, Jomari Ingol

Functional Requirements:

User Authentication:
The system will allow players to create an account and log in.

Rationale: Memory is tied to logins/accounts so traditionally it will follow the player across devices.

Structured Fact Storage:
a) The system will store explicit & unique player facts. For example: name, order history, hobbies in a relational database.

b) The system will also store NPC identity and personality data (example: NPC name, role, base personality) in a relational database.

Rationale: Provides exact recall of key events (example: "Player Name: harry pottah").

NPCs are persistent entities; the same NPC might exist across multiple sessions and players because this small world has limited NPC's.

Vector Embedding Storage:
The system converts each interaction into a vector embedding and then stores it in pgvector, each embedding is linked to both the Player ID and the NPC ID.
Rationale: These interactions to vectors will allow the system to do semantic search for emotionally related memories or conversations that could add meaning in the next interaction. Linking to both IDs ensures the right player-NPC memory pair is retrieved.

Context Retrieval:
Whenever the user has an interaction with our NPC's, the backend shall retrieve facts and perform a vector search for related past interactions.

Rationale: NPCs need relevant history which is tied to that user can affect how the players are treated differently.

LLM Prompt Construction:
The system's focus is to combine player input with retrieved context into a structured prompt which will turn into integration.

Rationale: LLM needs context to generate a memory-aware response, the previous or newly added data from the interactions is what the structure is built upon.

Primary LLM Integration:
The system with the retrieved context for the interaction then sends the prompt to an AI model (example: Deepseek or Gemini, set as a primary API).

Rationale: DeepSeek as a primary model is cost-efficient and handles natural conversation.

Backup LLM Fallback:
If Deepseek fails, the system will automatically retry with the Gemini API, the failed attempt will depend on the prompt or output.

Rationale: This outcome is very important as it ensures service continuity with the players and prevents game-breaking interruptions, the players expect the NPC to hold a conversation.

Dynamic Emotional Response:
The LLM response is meant to generate dialogue(s) reflecting the NPC's emotional state of the NPC's mode (example: mad, sad, you called his dog ugly).

Rationale: Core differentiator: NPCs react with emotion, not just facts.

Memory Update (Write-Back):
The system shall save the new interaction as both a fact and a new vector embedding. The interaction is linked to the specific Player ID and NPC ID pair.

Rationale: Without write-back, the NPC forgets the current interaction next session, breaking the core promise of persistent memory, and storage vectors. This ensures each NPC builds a separate memory of each player.

Browser Game Client:
The system provides an HTML5/JS browser-based game client for restaurant actions; the game is a small open-world. The restaurant setting is only for the NPC's and users to connect; it did not focus on the food transactions.

Rationale: Browser-based access removes friction for playtesters and presenters, allowing instant play without downloads or installations, as login accounts hold the persistent memory of the progress and data.

Non-Functional Requirements:

Usability:
The system will provide a consistent and easy-to-understand browser interface for player movement, NPC interaction, dialogue, and login functions.

Rationale: Since the game will be browser-based and used by players, the interface should be simple enough to understand without extensive instructions.

Performance:
The system should return NPC dialogue responses within approximately 5 seconds under normal network conditions. Player movement and basic game interactions should occur without noticeable delay.
Rationale: Long response times could interrupt conversations with NPCs and make the game feel unresponsive.

Database Performance:
The system should retrieve structured player facts and relevant vector memories quickly enough that database operations do not create noticeable interruptions during NPC conversations.
Rationale: Each NPC interaction depends on information being retrieved from PostgreSQL and pgvector before the prompt can be sent to the LLM.

Reliability:
The system will preserve player progress, NPC memories, structured facts, and interaction history between game sessions.
Rationale: Persistent memory is one of the main features of the project. Losing stored information would cause NPCs to forget previous interactions and would break the intended experience.

Availability / LLM Fallback:
The system will attempt to use a secondary LLM service if the primary LLM service becomes unavailable or fails to generate a usable response.
Rationale: Using DeepSeek as the primary model and Gemini as a backup reduces the chance that an external API failure completely prevents the player from interacting with NPCs.

Cross-Device Persistence:
Player memory and saved information will remain connected to the player's account so that stored information can be accessed when the player logs in from another supported device.
Rationale: The player should experience the same NPC memories and game progress regardless of which device is used to access the browser game.

Security:
The system will protect player account information, game data, and stored NPC memory from unauthorized access. Passwords, if stored by the application, will not be stored as plain text.
Rationale: User accounts are connected to persistent player data and NPC memories, so account information must be protected from unauthorized users.

Secure Communication:
Communication between the browser client, backend server, database, and external LLM services should use secure connections.
Rationale: Player input and account-related information may travel between several parts of the system and should be protected while being transmitted.

Input Validation:
The system will validate user input before it is stored in the database or used by backend services.
Rationale: Validating input helps prevent invalid or harmful data from affecting the database, game logic, or external AI requests.

Maintainability:
The system will separate the browser game client, backend logic, database operations, and LLM integration into organized components.
Rationale: Separating the main parts of the application will make debugging, testing, and future changes easier for the development team.

Extensibility:
The system should allow new NPCs, personality information, locations, memories, and game features to be added without requiring major changes to the existing application.
Rationale: The project may expand during development, and a flexible design will allow the team to add content without rebuilding major parts of the system.

Version Control / Supportability:
The source code for the system will be maintained using version-control software such as Git and GitHub.
Rationale: Version control will allow multiple team members to work on the project while tracking changes and reducing the risk of lost or conflicting code.

Browser Compatibility:
The game should function correctly in modern desktop web browsers supported by the development team.
Rationale: Because the application is an HTML5/JavaScript browser game, players should be able to access it without installing separate game software.

The Architecture of our application:

Our system has four main pieces working together. The browser client is the actual game built in HTML5 and JavaScript, and it handles login and all player interactions. When a player talks to

an NPC, that message goes to our backend server, which manages authentication, retrieves player memory, and builds the prompt sent to the AI model. For memory storage we use Postgres with the pgvector extension, which lets us keep one database that handles both structured facts, like specific past events, and vector embeddings for semantic search, so the system can find relevant memories even if they weren't stored in exact wording. The backend combines both types of memory into context, then sends that context along with the player's message to our LLM API, using DeepSeek as the primary model and Gemini as a backup. Once the model generates a response, the backend saves the new interaction back into the database as both a fact and an embedding, then sends the NPC's reply to the player. This means every interaction builds on the last, and memory is tied to the player's account, so it carries over across sessions and devices.

